RECORD TASK

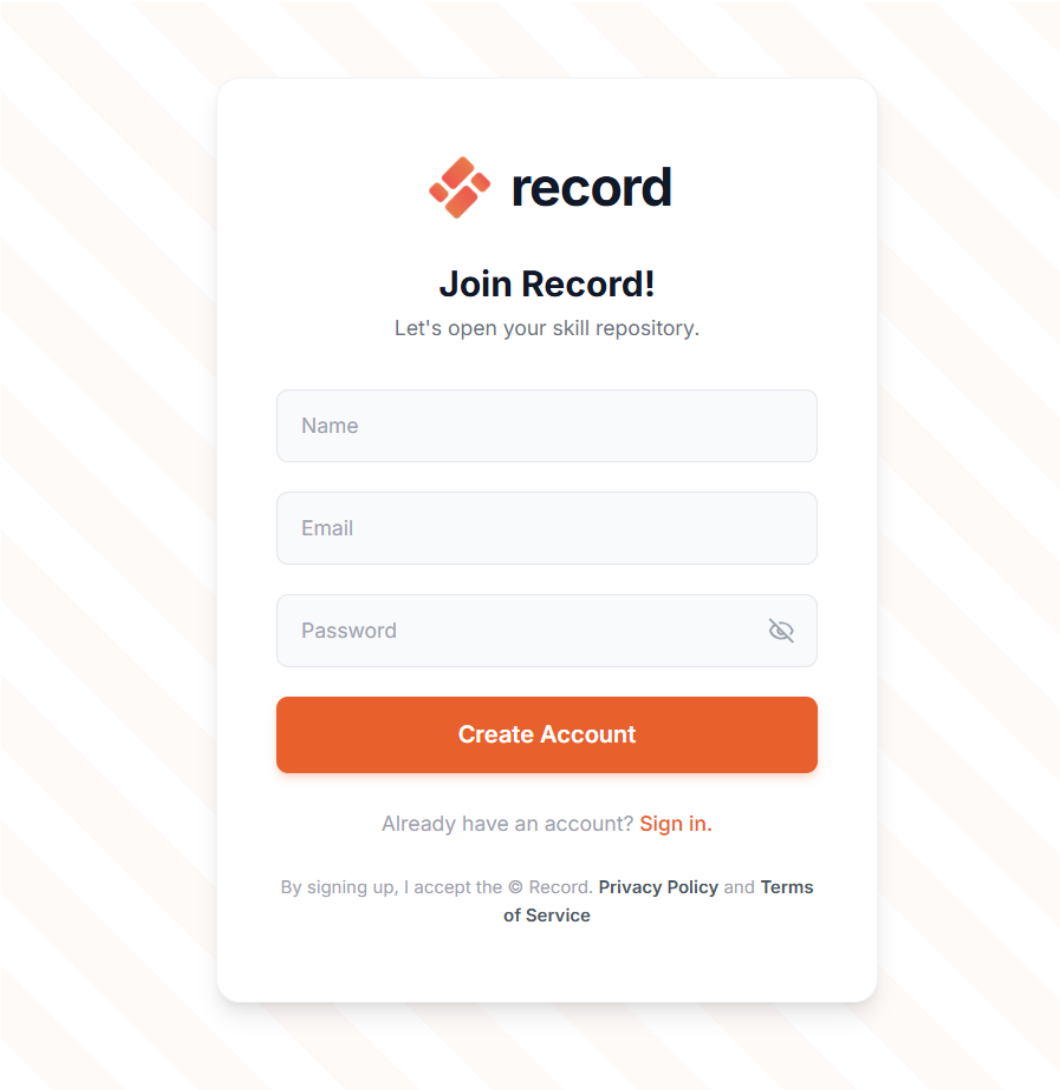
NAME : PRADEEP RP

DEPT : MSc.,DCS

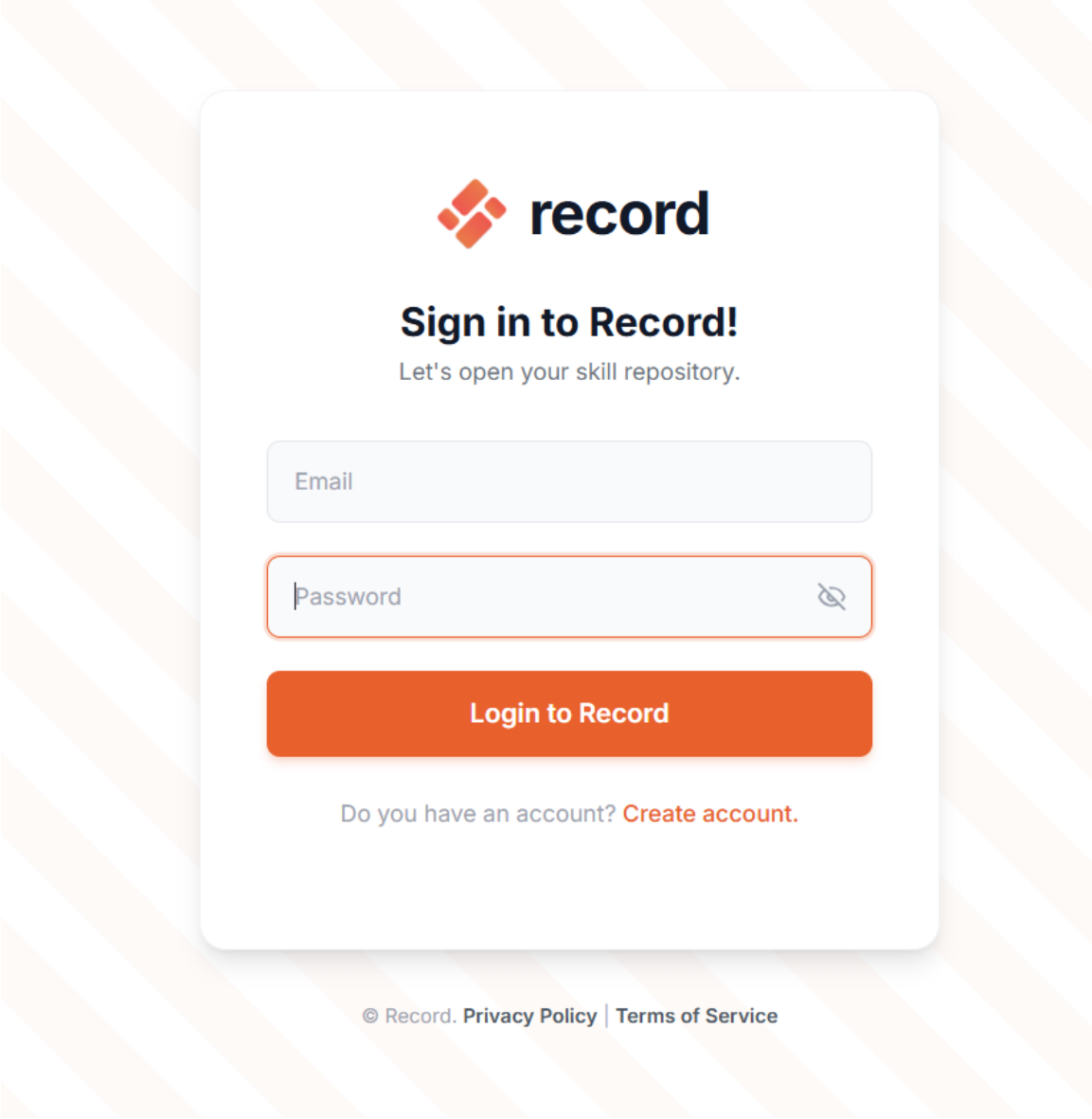
GIT HUB : https://github.com/PradeepRP08/record.git

OUTPUT :

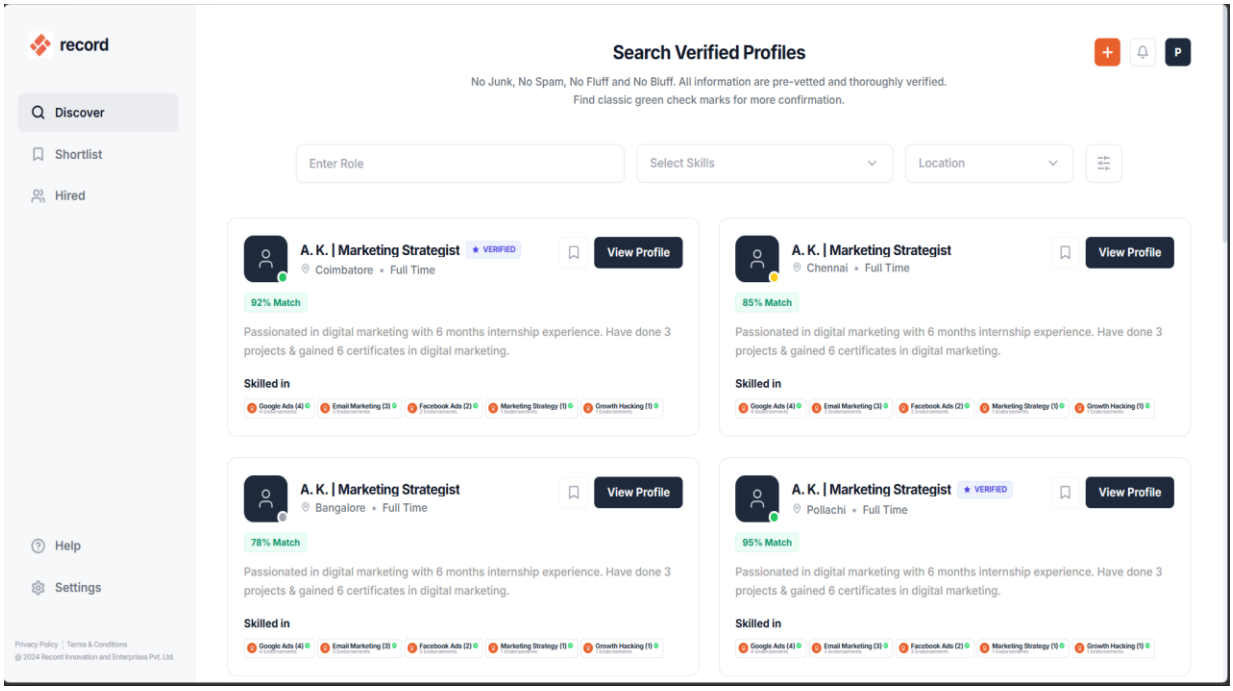
SIGN UP PAGE :

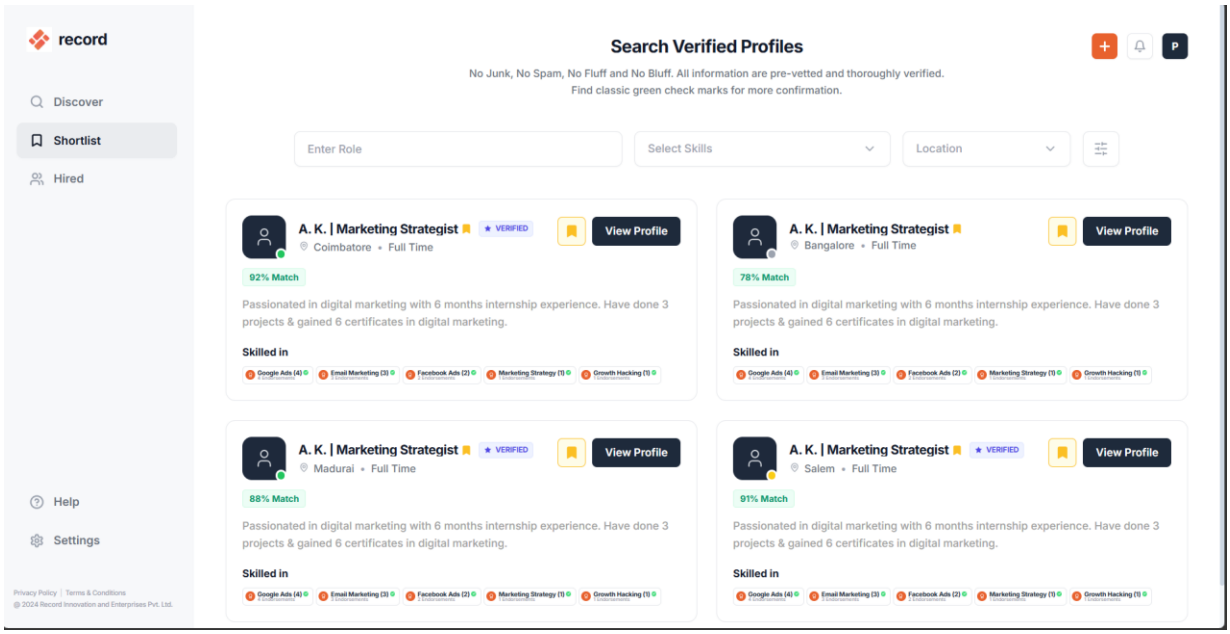


LOGIN PAGE :

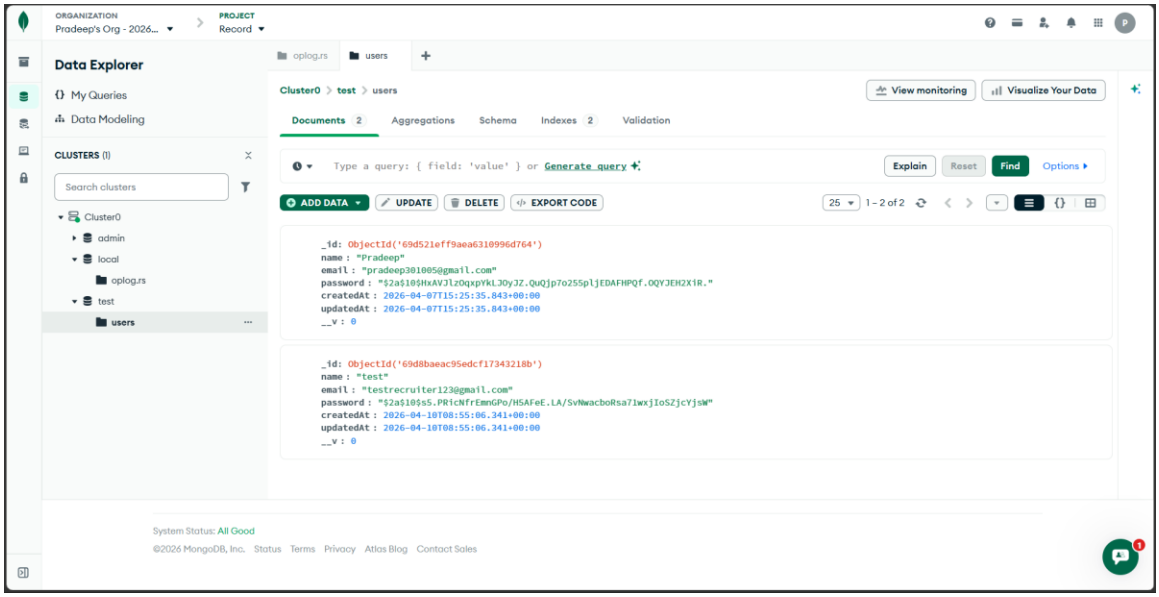

record
Sign in to Record!
Let's open your skill repository.
Email
Password
Login to Record
Do you have an account? Create account.
© Record. Privacy Policy | Terms of Service

DASHBOARD PAGE :





MONGODB :



The Record application is a full-stack MERN (MongoDB, Express.js, React, Node.js) platform designed to streamline the discovery, verification, and management of professional profiles.

Foundation & System Architecture

- The project was initialized with a strong architectural foundation using Vite for a fast and optimized React frontend and Node.js with Express for backend API services. This ensures high performance and modular scalability.
- A cloud-hosted MongoDB Atlas database was integrated using Mongoose ODM, enabling structured schema design, validation, and efficient querying.
- Security was a primary focus. Authentication was implemented using bcrypt for password hashing and JSON Web Tokens (JWT) for secure session management. Social logins were intentionally avoided to maintain full control over authentication logic.
- Frontend routing was handled using React Router, along with a custom ProtectedRoute component to restrict access to sensitive pages such as the dashboard and internal modules.

Frontend UI/UX Engineering

- The frontend was designed using Tailwind CSS with a focus on clean, modern aesthetics.
- A pixel-perfect responsive dashboard was developed by carefully replicating complex UI layouts. Search filters were aligned precisely with profile cards, and layout grids were optimized for both desktop and mobile views.
- UI elements such as skill badges were redesigned to be compact and non-overflowing. Spacing, typography, and component alignment were standardized across the application.
- Data visualization components were enhanced with high-contrast labels and dynamic rendering to ensure clarity and usability.
- User experience was significantly improved through micro-interactions, including React Hot Toast notifications, availability indicators, verified badges, and match percentage displays.
- Skeleton loading states were implemented to eliminate layout shifts during API calls, resulting in a smoother and more premium user experience.

Backend Services & Core Functionality

- The backend includes secure RESTful APIs for authentication and user management, including endpoints such as /api/auth/signup, /api/auth/login, and /api/auth/me.
- These endpoints support seamless frontend-backend communication and enable session persistence across page reloads.
- A dynamic NotificationService was developed to replace static alerts. This service listens to backend events and triggers real-time notifications for system activities, improving administrative visibility and responsiveness.

Testing & Code Quality Assurance

- To ensure enterprise-grade stability, a comprehensive testing framework was implemented.

- Frontend testing was conducted using Vitest and React Testing Library to validate component rendering, UI behavior, and authentication flows.
- Backend testing was implemented using Jest and Supertest to simulate HTTP requests and validate API responses, middleware security, and input handling.
- This testing infrastructure ensures reliability, prevents regressions, and supports continuous development.